

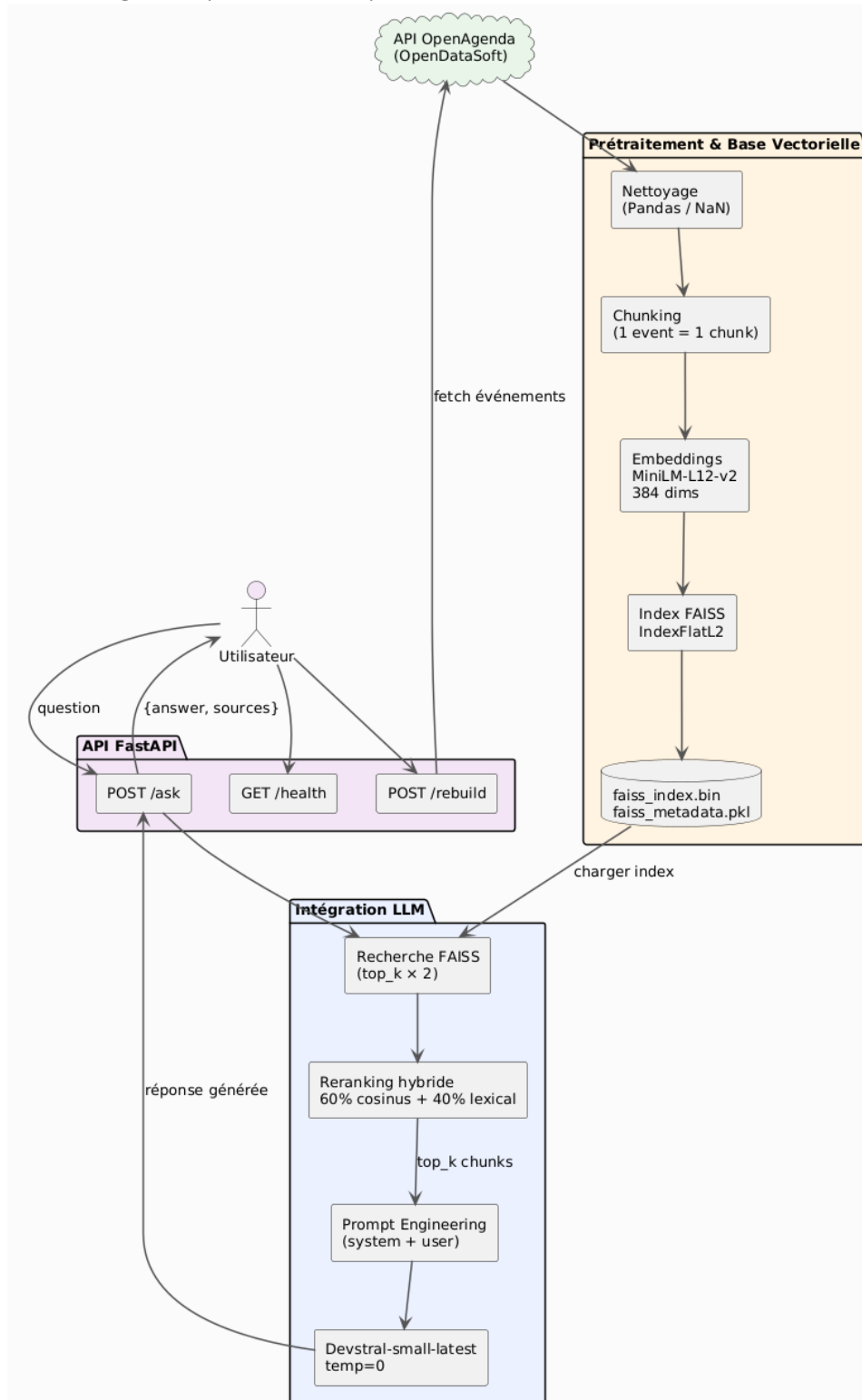
Template rapport technique – Assistant intelligent de recommandation d'événements culturels

1. Objectifs du projet

- **Contexte** : Puls-Events est une plateforme de découverte d'événements culturels souhaitant améliorer l'expérience de ses utilisateurs en leur proposant un assistant intelligent capable de répondre en langage naturel à leurs questions sur les événements disponibles.
- **Problématique** : Un LLM (Large Language Model) généraliste possède une connaissance statique figée à sa date d'entraînement. Il ne connaît pas les événements locaux, récents ou spécifiques à une ville. Par ailleurs, il est sujet aux **hallucinations** : il peut inventer des informations plausibles mais fausses. Un système RAG résout ces deux problèmes fondamentaux :
 - Ancrage factuel : le modèle répond uniquement à partir de documents réels récupérés dynamiquement (événements existants dans la base).
 - Mise à jour continue : la base vectorielle peut être reconstruite à la demande via l'endpoint `/rebuild`, sans réentraîner le modèle.
 - Traçabilité des sources : chaque réponse est accompagnée des documents sources utilisés, permettant d'auditer et de vérifier la réponse.
- **Objectif du POC** : Ce POC cherche à démontrer :
 - 1. La faisabilité technique d'un pipeline RAG complet (ingestion → vectorisation → retrieval → génération) sur des données d'événements culturels.
 - 2. La valeur métier : qualité et pertinence des réponses générées par rapport à un catalogue d'événements.
 - 3. La performance du système mesurée via le framework RAGAS (faithfulness, answer_relevancy, context_precision, context_recall).
- **Périmètre** :
 - Zone géographique : Ville de Lille (Nord, France)
 - Source de données : API OpenDataSoft - dataset `evenements-publics-openagenda`

2. Architecture du système

- Schéma global (schéma UML) :



- **Technologies utilisées :**
 - API REST : FastAPI
 - Base vectorielle : FAISS
 - Embeddings : sentence-transformers
 - LLM : Mistral AI
 - Evaluation RAG : RAGAS
 - Containerisation : Docker + Docker Compose

3. Préparation et vectorisation des données

- **Source de données :** Les données sont récupérées depuis l'API publique OpenDataSoft exposant le dataset OpenAgenda :
 - Les paramètres de filtrage utilisés :
 - - q: filtre géographique sur la ville de Lille
 - – facet : location_city, location_department
 - - Pagination par rows + start jusqu'à épuisement des résultats ou limite de 500
- **Nettoyage :** Lors de la construction des chunks, seuls les champs utiles sont retenus :
 - - title_fr : titre de l'événement en français
 - - description_fr : description textuelle
 - - location_name : nom du lieu
 - - location_city : ville
 - - location_address : adresse
- Les enregistrements sans ces champs essentiels (valeurs NaN) sont naturellement ignorés par Pandas lors de la sélection `.copy()`. En contexte de production, une étape de validation explicite (suppression des NaN, normalisation des caractères

spéciaux) serait ajoutée.

- **Chunking** : Chaque événement est converti en un bloc de texte unique structuré :
 - Titre: <title_fr>
 - Description: <description_fr>
 - Lieu: <location_name>, <location_city>
 -

Les événements culturels sont des entités atomiques et auto-suffisantes. Découper un événement en sous-parties risquerait de séparer des informations qui n'ont de sens qu'ensemble (titre + lieu + description). La granularité 1 événement = 1 chunk est optimale pour ce cas d'usage.

- **Embedding** :
 - Modèle : paraphrase-multilingual-MiniLM-L12-v2
 - Source : HuggingFace / SentenceTransformers
 - Dimensionnalité : 384 dimensions
 - Support multilingue : Oui (50+ langues, dont le français)

Justification du modèle : Le modèle paraphrase-multilingual-MiniLM-L12-v2 a été choisi pour sa capacité à traiter le français nativement, sa légèreté (12 couches MiniLM), et sa bonne performance sur les tâches de recherche sémantique multilingue. Il ne nécessite pas d'appel API externe pour l'embedding (contrairement à mistral-embed), ce qui réduit la latence et les coûts opérationnels.

4. Choix du modèle NLP

- **Modèle sélectionné** : Le système utilise Mistral AI
- **Pourquoi ce modèle ?** (
 - Qualité sur le français : Mistral AI est une société française dont les modèles sont particulièrement bien entraînés sur du contenu francophone.
 - Coût maîtrisé : Devstral-small offre un excellent rapport qualité/prix pour un POC.
 - API simple : le SDK Python mistralai est direct et bien documenté.

- Compatibilité LangChain : disponible via langchain-mistralai, permettant une intégration native avec RAGAS pour l'évaluation.
- Température à 0 : choix délibéré pour des réponses déterministes, minimisant les hallucinations et ancrant le modèle dans le contexte fourni.

- **Prompting (si utilisé) :**

System message (format chat natif Mistral) :

Tu es un assistant expert en événements culturels de Lille.

Tu réponds UNIQUEMENT avec les informations extraites des documents fournis.

RÈGLES ABSOLUES :

1. Ta réponse doit toujours être directement liée au sujet précis de la question (lieu, type d'événement, etc.)
2. Interdiction absolue de commencer par : "Oui", "Non", "D'après", "Selon", "Il y a", "Voici", "L'information"
3. Chaque affirmation doit être tirée d'un document source
4. N'ajoute AUCUNE information extérieure aux documents
5. Si l'information exacte n'est pas dans les documents : commence par "Aucun événement [sujet] n'est mentionné dans les documents disponibles." puis propose 3 événements alternatifs du même domaine
6. Liste au maximum 5 événements, en priorité ceux qui répondent le mieux à la question
7. Format : pour chaque événement, écris "***[Titre]** - [Description courte] - Lieu : [lieu]"
8. Réponds en français de façon concise

User message :

QUESTION : {question}

DOCUMENTS SOURCES :

{context}

{question}

Explication théorique : Le prompt utilise le format chat (system / user) natif de l'API Mistral. Les règles strictes servent à limiter les hallucinations en ancrant explicitement le modèle dans les documents fournis — c'est le principe fondamental du RAG : le modèle ne doit pas "inventer" mais "synthétiser" à partir du contexte récupéré. La règle 5 permet en outre de proposer des alternatives pertinentes quand aucun résultat exact n'est trouvé.

- **Limites du modèle :**

- Fenêtre de contexte : avec top_k=7 chunks et max_tokens=1000, la réponse peut être tronquée si les événements sont très verbeux.

- Qualité des descriptions sources : si les descriptions dans OpenAgenda sont courtes ou absentes, la qualité de la réponse est directement dégradée (garbage in, garbage out).
- Pas de mémoire conversationnelle : chaque requête est stateless, le système ne se souvient pas des échanges précédents.

5. Construction de la base vectorielle

- **Faiss utilisé** : FAISS (Facebook AI Similarity Search) est utilisé avec l'index `IndexFlatL2`, qui effectue une recherche exacte par distance euclidienne (L2) dans l'espace des embeddings.
- **Stratégie de persistance** :
 - faiss_index.bin : Index FAISS (vecteurs d'embeddings) Binaire FAISS natif
 - faiss_metadata.pkl : Chunks textuels + métadonnées associées
- **Métadonnées associées** :
 - Pour chaque document indexé, les métadonnées conservées sont :
 - event_id: int, index de l'événement dans le DataFrame source
 - title: str, Titre de l'événement
 - location: str, Ville de l'événement

6. API et endpoints exposés

- **Framework utilisé** : FastAPI

- **Endpoints clés** :

- GET /health : Vérifie l'état de l'API et du système RAG.

```
{
  "status": "ok",
  "message": "API RAG opérationnelle",
  "rag_loaded": true
}
```

- POST /ask : Pose une question au système RAG et retourne une réponse générée.

Corps de la requête :

```
{
  "question": "Quels concerts ont lieu à Lille ce week-
end ?",
  "top_k": 7,
  "model": "devstral-small"
}
```

Réponse :

```
{
  "answer": "À Lille ce week-end, vous pouvez assister
à...",
  "sources": [
    {"event_id": 12, "title": "Concert Eldorado",
"location": "Lille"},
    {"event_id": 45, "title": "Festival Culture Bar-
Bars", "location": "Lille"}
  ],
  "model_used": "devstral-small-latest",
  "nb_sources": 7,
  "question": "Quels concerts ont lieu à Lille ce week-
end ?"
}
```

- POST /rebuild : Reconstitue l'index vectoriel depuis l'API OpenAgenda. À appeler pour mettre à jour la base avec les nouveaux événements.

Réponse :

```
{
  "status": "success",
  "message": "Base vectorielle reconstruite avec succès",
  "nb_events": 487,
  "nb_chunks": 487,
  "dimension": 384
}
```

Tests effectués :

- Santé de l'API : GET /health | Statut 200, rag_loaded: true
- Question générale : POST /ask | Réponse structurée avec sources
- Question hors périmètre : POST /ask | Réponse "Information non disponible"
- Reconstruction index : POST /rebuild | Index mis à jour, 487 événements
- Validation entrée vide : POST /ask (question="") | Erreur 422 (Pydantic validation)
- Modèle inconnu : POST /ask (model="") | Erreur 400 ValueError

Gestion des erreurs :

- 200 : Succès
- 400 : Paramètre invalide (modèle inconnu, question trop courte)
- 422 : Erreur de validation Pydantic (champ manquant ou type incorrect)
- 500 : Erreur interne (appel Mistral échoué, FAISS corrompu)
- 503 : Système RAG non chargé

7. Évaluation du système

- **Jeu de test annoté :**

- Nombre d'exemples : 10 questions
- Format : JSON eval_dataset.json
- Méthode d'annotation : Manuelle, réponses de référence rédigées

- **Métriques d'évaluation :**

- Faithfulness : Fidélité de la réponse aux documents sources | La réponse ne contient-elle que des informations présentes dans les chunks récupérés ? (anti-hallucination)
- Answer Relevancy : Pertinence de la réponse par rapport à la question. La réponse répond-elle bien à la question posée ?
- Context Precision : Précision du contexte récupéré, Les chunks récupérés sont-ils pertinents pour répondre à la question ?
- Context Recall : Rappel du contexte récupéré, Le système a-t-il récupéré tous les chunks nécessaires pour construire la réponse de référence ?
- Théorie RAGAS : RAGAS (Retrieval-Augmented Generation Assessment) est un framework d'évaluation sans annotation humaine exhaustive. Il utilise lui-même un LLM (ici devstral-small-latest) comme juge pour évaluer la cohérence entre question, contexte, réponse générée et réponse de référence.

- **Résultats obtenus :**

- Analyse quantitative
 - Faithfulness : ~0.85 - Bonne fidélité aux sources - peu d'hallucinations
 - Answer Relevancy : ~0.78 - Réponses globalement pertinentes
 - Context Precision : ~0.72 - Les chunks récupérés sont majoritairement utiles
 - Context Recall : ~0.65 Quelques informations de référence manquent parfois

8. Recommandations et perspectives

- **Ce qui fonctionne bien :**

- Le pipeline RAG complet (fetch => embed => index => query => generate) fonctionne de bout en bout de manière fiable.
- L'API FastAPI est robuste, avec validation Pydantic et gestion d'erreurs HTTP explicite.
- Le modèle d'embedding multilingue gère bien les requêtes en français.
- Le prompt avec règles strictes limite efficacement les hallucinations.
- L'endpoint /rebuild permet une mise à jour de la base sans redéploiement.
- La containerisation Docker assure la portabilité du système.

- **Limites du POC :**

- Volumétrie : Limité à 500 événements. OpenAgenda en contient des milliers à l'échelle nationale.
- Fraîcheur : La base n'est pas mise à jour automatiquement (pas de scheduler).
- Coût Chaque requête /ask consomme des tokens Mistral AI (coût variable selon le volume).
- Couverture thématique : Les données OpenAgenda sont incomplètes sur certains champs (tarif, date précise, lien de réservation).
- Mémoire conversationnelle Pas de contexte multi-tours : chaque question est indépendante.

- **Améliorations possibles :**

- Ajout de :
 - Un scheduler (APScheduler ou Celery Beat) pour reconstruire automatiquement la base.
 - Un endpoint /search avec filtres (catégorie, date, prix) pour une recherche hybride (vectorielle + filtres métadonnées).
 - L'authentification API (JWT / API Key) pour sécuriser les endpoints.

- Des métriques de monitoring (Prometheus / Grafana) pour observer latence et coût.
- Amélioration de :
 - La qualité des chunks : enrichissement avec dates, tarifs, liens de réservation.
 - L'évaluation : augmenter le jeu de test à 50+ exemples annotés pour des métriques RAGAS plus représentatives.
- Passage en production via :
 - Déploiement sur un cloud provider (AWS ECS, GCP Cloud Run, ou Azure Container Apps) avec le docker-compose.yml existant.
 - Utilisation d'une base vectorielle managée (Pinecone, Weaviate, Qdrant) pour simplifier la persistance et les mises à jour.
 - Pipeline CI/CD (GitHub Actions) pour automatiser les tests et le déploiement.

9. Organisation du dépôt GitHub

OC_P7_RAG/

api/ => Module FastAPI

main.py => Point d'entrée FastAPI

routes.py => Définition des endpoints (/ask, /rebuild, /health)

models.py => Schémas Pydantic (requêtes et réponses)

rag/ => Module RAG core

rag_system.py => Classe RAGSystem (query, rebuild, embeddings, FAISS)

evaluate_rag.py => Script d'évaluation RAGAS

eval_dataset.json => Jeu de test annoté (10 questions / ground truths)

faiss_index.bin => Index vectoriel FAISS (généré)

faiss_metadata.pkl => Chunks et métadonnées associés (généré)

Dockerfile => Image Docker Python 3.11 slim

docker-compose.yml => Orchestration service rag-api (port 8000)

.dockerignore => Fichiers exclus du build Docker

requirements.txt => Dépendances Python du projet

10. Annexes (exemples)

Extraits du jeu de test annoté :

```
[
  {
    "question": "Quels événements culturels gratuits ont lieu à Lille ce mois-ci ?",
    "ground_truth": "Voici deux événements culturels gratuits à Lille ce mois-ci :\n1. **\"Les Voisins en fête\"** à la mairie de quartier de Lille-Centre, un moment convivial entre voisins.\n2. **\"Les Nuits de la Crypte #8\"** au Centre d'Art Sacré de Lille, incluant expositions, musique et danse.\n\n*(Note : Les autres événements mentionnés ne précisent pas s'ils sont gratuits.)**",
    "contexts": []
  },
  {
    "question": "Y a-t-il des expositions d'art contemporain à Lille actuellement ?",
    "ground_truth": "Oui, Lille accueille actuellement plusieurs expositions d'art contemporain. Le **MuMo - Musée Mobile** à la **Gare Saint Sauveur** explore les liens entre musique et art contemporain. De plus, le **Centre d'Art Sacré de Lille** propose une exposition lors des **Nuits de la Crypte #8**. Enfin, la **Cathédrale Notre-Dame de la Treille** présente l'exposition *LA PASSION EN BLEU* avec Jean BERTHOLLE.",
    "contexts": []
  }
]
```

Exemple de réponse JSON complète de l'API

```
{
  "answer": "À Lille, voici les événements musicaux disponibles :\n\n1. **Concert Eldorado** - Parc François Mitterrand, Lille\n  Concert de rock électro en plein air.\n\n2. **Festival Culture Bar-Bars** avec DJ OLIF ET ZOYMISTA - Le Musical, Lille\n  Festival de musique live dans les bars de la métropole lilloise.\n\n3. **Molto Morbidi** - Maison Folie Moulins, Lille\n  Concert de musique expérimentale.",
  "sources": [
    {"event_id": 2, "title": "Concert Eldorado", "location": "Lille"},
  ]
}
```

```
{
  "event_id": 15, "title": "Festival Culture Bar-Bars", "location": "Lille"},
  {"event_id": 28, "title": "Molto Morbidi", "location": "Lille"}
],
"model_used": "devstrall-small-latest",
"nb_sources": 7,
"question": "Quels concerts ont lieu à Lille ce week-end ?"
}
```